INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

APLICACIONES PARA COMUNICACIONES EN RED

Reporte

“Control de Flujo”

Grupo: 6CM2

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Moreno Cervantes Axel Ernesto

Fecha de entrega: 15 de octubre de 2025

ÍNDICE

INTRODUCCIÓN 2
DESARROLLO 4
1.- Archivo: Servidor.py 4
2.- Archivo: Cliente.py 8
3.- Archivo: Reproductor.py 12
CONCLUSIÓN 14

INTRODUCCIÓN

Biblioteca “pygame”

Pygame es una biblioteca multimedia para Python orientada a videojuegos y aplicaciones interactivas. Incluye módulos para gráficos, entrada y audio; el audio se gestiona principalmente con *pygame.mixer* y *pygame.mixer.music.*

Características principales

- Soporta reproducción “streaming” de música (ideal para archivos largos como MP3)
- También tiene objetos Sound para efectos cortos (mejor con WAV).
- La capacidad real de reproducir MP3 depende de cómo se compiló/instaló SDL_mixer en tu sistema. En versiones recientes de pygame la compatibilidad MP3 mejoró, pero en algunos sistemas sigue ser menos fiable que OGG/WAV.

Comandos / Funciones para reproducir MP3

Comando / Función	Descripción
pygame.mixer.init()	Inicializa el subsistema de audio. Conviene llamarlo antes de usar mixer.
pygame.init()	Inicializa varios subsistemas incluyendo audio.
pygame.mixer.music.load("ruta")	Carga un archivo de audio (ideal para MP3 largos). No lo reproduce aún, solo lo deja listo para reproducir.
pygame.mixer.music.play(loops, start)	Inicia la reproducción del archivo cargado. loops=0 (suena una vez), loops=-1 (bucle infinito), start indica el segundo inicial.
pygame.mixer.music.pause()	Pausa la reproducción (se puede reanudar luego).
pygame.mixer.music.unpause()	Reanuda una reproducción previamente pausada.
pygame.mixer.music.stop()	Detiene la reproducción y resetea la posición al inicio.
pygame.mixer.music.set_volume(value)	Ajusta volumen de la música (valor de 0.0 a 1.0).
pygame.mixer.music.get_busy()	Devuelve True si la música está sonando o en pausa.
pygame.mixer.music.queue(path)	Encola otro archivo para que suene justo después del que está sonando.
pygame.mixer.music.set_pos(pos)	Salta a una posición (segundos) dentro del archivo si el backend lo permite.
pygame.mixer.quit()	Cierra el subsistema de audio cuando ya no lo necesitas.

Modulo “playsound3”

playsound3 es una biblioteca muy pequeña cuyo único objetivo es reproducir sonidos con la menor fricción posible. Se instala con pip install playsound3 y delega en el reproductor/soporte del sistema para hacer la reproducción.

Características principales

- **API mínimo:** una función principal playsound() que reproduce un archivo (o URL).
- **Multiplataforma:** funciona en Windows/macOS/Linux usando utilidades nativas del OS o binarios disponibles.
- **Bloqueante por defecto:** la llamada espera hasta que termine la reproducción.

Comandos / Funciones para MP3

Comando / Función	Descripción
from playsound3 import playsound	Importa la función principal para reproducir audio.
playsound(path, block)	Reproduce el archivo indicado. • block=True hace que la llamada bloquee hasta que termine la reproducción. • En versiones modernas block=False intenta reproducir en segundo plano y puede devolver un objeto reproductor.
s = playsound(path, block=False)	Devuelve un objeto controlador/worker y permite consultar/gestionar la reproducción.
s.is_alive()	Consulta si la reproducción en segundo plano aún sigue activa.
s.stop()	Intenta detener la reproducción en segundo plano.

Limitaciones

- La API intencionalmente mínima, pues playsound está pensada para “poner a sonar algo” sin control avanzado.
- Depende del reproductor o utilidades del sistema; si el sistema no tiene soporte para MP3 por consola, playsound3 puede fallar con archivos MP3.

Socket de datagrama

Un socket de datagrama es un extremo de comunicación que envía y recibe mensajes independientes (datagramas) a través de la red. En la práctica está ligado al protocolo UDP y su característica principal es ser orientado a mensajes y sin conexión.

Propiedades

- **Orientado a mensajes:** cada envío transmite una unidad independiente (un datagrama). Los límites del mensaje se conservan; si envías 3 KB en un send, el receptor recibe exactamente ese paquete (o nada), no un flujo continuo de bytes.

- **Sin conexión:** no hay "handshake" ni conexión establecida antes de enviar; el emisor especifica la dirección destino para cada datagrama (aunque algunos stacks permiten una connect opcional para fijar destino).
- **No confiable / no ordenado (por defecto):** los datagramas pueden perderse, llegar duplicados o fuera de orden; el protocolo no garantiza retransmisión ni reordenamiento.
- **Bajo overhead y baja latencia:** sin gestión de conexión ni control de flujo complejo, tienen menos latencia y menor overhead que conexiones orientadas (como TCP).
- **Tamaño limitado / fragmentación:** hay límites prácticos (MTU). Si un datagrama excede el MTU, la red lo fragmentará; eso puede causar pérdidas parciales o penalizar rendimiento. Mejor enviar mensajes pequeños y manejar fragmentación a nivel de aplicación si es crítico.
- **Bloqueo:** en modo bloqueante, la operación de recibir espera hasta que llegue un datagrama; enviar suele retornar rápido cuando el datagrama queda en la cola del sistema.

DESARROLLO

Para nuestro reproductor de mp3 se crearon tres archivos: Servidor.py, Cliente.py y Reproductor.py, donde el último archivo corresponde a la interfaz gráfica del reproductor, aquel con el que podemos controlar la reproducción.

1.- Archivo: Servidor.py

- **listarCanciones()**

```python
import socket, struct
from  pathlib import Path

SERVER_ADDR = ("0.0.0.0", 5000)
MAX_BYTES_PAQUETE = 1024
ENCABEZADO_FORMAT = "!I"
ENCABEZADO_SIZE = 4

def listarCanciones() -> list[str]:
    pistas = Path(__file__).parent/"pistas"

    if (not pistas.exists()):
        return []

    canciones = [f.name for f in pistas.glob("*.mp3")]

    return canciones
```

Imagen 1. Creación del sockect y la función listarCanciones().

Para este bloque de código, tenemos las siguientes variables globales:

- **SERVER_ADDR = ("0.0.0.0", 5000):** Es una tupla con la dirección IP y puerto donde escucha el servidor.
- **MAX_BYTES_PAQUETE = 1024:** Es el tamaño máximo en bytes del contenido útil dentro de cada paquete.

- **ENCABEZADO_FORMAT = "!I":** Es el formato del encabezado del paquete. Se usa para convertir el número de secuencia (número de paquete) a bytes.
- **ENCABEZADO_SIZE = 4:** Es el tamaño en bytes del encabezado del paquete.

También tenemos la función **'listarCanciones()'**, que se encarga de obtener la ruta de las canciones, obtener sus nombres y guardarlos en una lista que se devuelve al final.

- **enviarCancion_gbn(sock: socket.socket, cliente_addr: tuple, filepath: Path, window_size=6)**

```python
def enviarCancion_gbn(sock: socket.socket, cliente_addr: tuple, filepath: Path, window_size=6) -> None:
    tamano_archivo = filepath.stat().st_size
    total_paquetes = (tamano_archivo + MAX_BYTES_PAQUETE - 1) // MAX_BYTES_PAQUETE

    # Se lee la canción en memoria
    with open(filepath, "rb") as file:
        bytes_archivo = file.read()

    print(f"\n>> Se envía al cliente: '{filepath.name}' ({tamano_archivo} bytes) en {total_paquetes} paquetes a {cliente_addr}")

    first_num_seq_ACKed = 0
    sgt_num_seq = 0

    sock.settimeout(360)
```

Imagen 2. Se obtiene el tamaño del archive y el número de paquetes.

Descripción general: Esta función implementa el algoritmo Go-Back-N para el envió de la canción al cliente usando sockets de datagrama bloqueantes.

Parámetros:

- **sock : socket.socket**
  - Socket UDP ya creado y enlazado.
- **cliente_addr : tuple**
  - Tupla (id, puerto) del cliente destino
- **filepath : Path**
  - Ruta del archivo .mp3 a enviar.
- **window_size : int**
  - Número máximo de paquetes "en vuelo".

En este bloque de código se obtiene el tamaño total de la canción a enviar y se guarda en la variable **'tamano_archivo'**. También se calcula el número de paquetes que se tendrán que enviar al cliente para poder enviar la canción completa, esto se guarda en la variable **'total_paquetes'**.

Después se lee la canción en memoria y se definen dos variables más. La variable **'first_num_seq_ACKed'**, es el número de secuencia del primer paquete no confirmado por el cliente y la variable **'sgt_num_sq'** es el número de secuencia del siguiente paquete a enviar.

Y por último se establece el tiempo de espera para recibir la confirmación del paquete por parte del cliente.

```python
    # Enviar nombre, tamaño y número total de paquetes del archivo (informativo)
    meta_datos = f"FILEINFO|{filepath.name}|{tamano_archivo}|{total_paquetes}"
    sock.sendto(meta_datos.encode(), cliente_addr) # Se envían los meta datos al cliente
```

Imagen 3. Envió de metadatos al cliente.

En estas líneas de código se le envía al cliente los metadatos de la canción (archivo), como el nombre de la canción, el tamaño de la canción y el número de paquetes que se van a enviar.

```python
    # Bucle principal para el envió de la canción (Algoritmo Go-Back-N)
    while (first_num_seq_ACKed < total_paquetes):
        # Enviar paquetes dentro de la ventana
        while ((sgt_num_seq < first_num_seq_ACKed + window_size) and (sgt_num_seq < total_paquetes)):
            # Para cada número de secuencia siguiente se calcula los siguiente
            inicio = sgt_num_seq * MAX_BYTES_PAQUETE
            final = inicio + MAX_BYTES_PAQUETE
            bytes_paquete = bytes_archivo[inicio:final]

            encabezado = struct.pack(ENCABEZADO_FORMAT, sgt_num_seq)

            paquete = encabezado + bytes_paquete

            sock.sendto(paquete, cliente_addr)
            print(f">> Se envía el paquete: {sgt_num_seq}")

            sgt_num_seq += 1
```

Imagen 4. Creación del paquete a enviar al cliente.

Este bloque de código corresponde el algoritmo Go-Back-N, en donde mientras el número de secuencia del primer paquete no confirmado sea menor que el número total de paquetes, es decir, mientras aun falten paquetes por enviar entonces se calcula que porción del archivo de la canción se enviará en el siguiente paquete, así como su encabezado con el número de secuencia correspondiente y por último se suman ambas cosas para formar el paquete a enviar.

Una vez enviado el paquete, entonces en número de secuencia se actualiza para el siguiente paquete.

```python
        try:
            data, addr = sock.recvfrom(4096)

            if (addr != cliente_addr):
                continue

            if (data.startswith(b"ACK")):
                ACK_seq = struct.unpack(ENCABEZADO_FORMAT, data[3:3 + ENCABEZADO_SIZE])[0]

                if (ACK_seq >= first_num_seq_ACKed):
                    first_num_seq_ACKed = ACK_seq + 1

                    print(f">> Se confirma paquete [{ACK_seq}] -> último paquete sin confirmar [{first_num_seq_ACKed}]")
            elif (data.startswith(b"STOP")):
                print(">> El cliente solicitó detener la transferencia")

                return
        except socket.timeout:
            print(f">> Perdida del paquete: Se reenvia desde el paquete [{first_num_seq_ACKed}]")

            sgt_num_seq = first_num_seq_ACKed

            continue

    print("\n>> Transferencia de la canción completa")

    sock.sendto(b"FIN", cliente_addr)
```

Imagen 5. Recibir confirmaciones del cliente.

En este bloque de código se espera la confirmación del cliente de que el paquete se envió correctamente. Primero se trata de obtener la respuesta del cliente, para ello verificamos que este provenga del cliente con el que actualmente nos estamos comunicando, por lo que comparamos la dirección del cliente con la dirección de la respuesta de este, si no coinciden entonces la respuesta se ignora.

Si la respuesta corresponde al mismo cliente, entonces verificamos si esta es la confirmación del envió correcto del paquete. Si es el caso, entonces se extrae del encabezado el número de secuencia del paquete confirmado y se verifica si corresponde con el número de secuencia del primer paquete no confirmado o si es de un paquete posterior. Si es el caso entonces actualizamos este último al número de secuencia del siguiente paquete por confirmar.

Si la respuesta corresponde a la solicitud del cliente para detener la transferencia de la canción, entonces nos salimos de la función y la transmisión se trunca.

Si el tiempo de espera para recibir la confirmación del paquete se excede, entonces actualizamos el número de secuencia del siguiente paquete por el del último paquete no confirmado, es decir, que se enviaran los paquetes desde el último no confirmado.

Si al final, todos los paquetes fueron enviados y confirmados, entonces la transferencia de la canción se completó.

- **main()**

```python
def main() -> None:
    # Se crea y se bindea un socket UDP
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(SERVER_ADDR)

    print(f">> Servidor Go-Back-N escuchando en {SERVER_ADDR}")

    while (True):
        data, addr = sock.recvfrom(4096)
        mensaje = data.decode(errors="ignore")

        if (mensaje == "LIST"): # Cliente solicita la lista de canciones
            lista_canciones = listarCanciones()
            bytes_paquete = "::".join(lista_canciones)

            sock.sendto(bytes_paquete.encode(), addr) # Se envía la lista de canciones

            print(f">> Lista enviada a {addr} ({len(lista_canciones)} canciones en total)")
        elif (mensaje.startswith("GET:")): # Cliente solicita la canción
            nombre_cancion = mensaje.split(":", 1)[1] # Se obtiene el nombre de la canción solicitada
            ruta_cancion = Path(__file__).parent/"pistas"/nombre_cancion # Se obtiene la ruta de la canción

            if (not ruta_cancion.exists()): # Si la canción no existe
                sock.sendto(b"ERROR|NOFILE", addr)

                print(f"Cliente pidió '{nombre_cancion}' pero no existe")

                continue

            # Enviar el archivo con la canción
            enviarCancion_gbn(sock, addr, ruta_cancion, window_size=6)
        else:
            # Mensaje desconocido
            sock.sendto(b"ERROR|UNKNOWN", addr)

if __name__ == "__main__":
    main()
```

Imagen 6. Respuesta a las solicitudes del cliente.

Este bloque de código corresponde a la función principal, en donde se crea y se bindea el socket UDP del servidor.

Mientras se sigan recibiendo solicitudes del cliente, se verifica si corresponde a la solicitud para el envió de la lista de canciones o a la solicitud del envío de la canción. Para el segundo caso se extrae de la solicitud el nombre de la canción para obtener su ruta y llamar a la función **'enviarCancion_gbn()'** para hacer el envío correspondiente. Si se da el caso en el que no se obtiene la ruta de la canción solicitada, entonces se envía el mensaje de error correspondiente.

Si la solicitud no corresponde a ninguno de los casos anteriores, entonces la solicitud se desconoce, por lo que se envía el mensaje de error correspondiente.

2.- Archivo: Cliente.py

- **pedirLista(sock: socket.socket)**

```python
import socket, struct, tempfile
from pathlib import Path
from Reproductor import reproductor

SERVER = ("127.0.0.1", 5000)
MAX_BYTES_PAQUETE = 1024
ENCABEZADO_FORMAT = "!I"
ENCABEZADO_SIZE = 4

def pedirLista(sock: socket.socket) -> list[str]:
    sock.sendto(b"LIST", SERVER)
    data, _ = sock.recvfrom(65536)
    canciones = data.decode().split("::") if data else []
    lista_canciones = [s for s in canciones if s]

    return lista_canciones
```

Imagen 7. Creación del socket del cliente y la función pedirLista().

Descripción general: Esta función envía la solicitud al servidor para recibir la lista de canciones y una vez recibida la lista, la devuelve.

Parámetros:

- **sock : socket.socket**
  - Socket UDP ya creado y enlazado.

Para este bloque de código, tenemos las siguientes variables globales:

- **SERVER_ADDR = ("127.0.0.1", 5000):** Es una tupla con la dirección IP y puerto del servidor.
- **MAX_BYTES_PAQUETE = 1024:** Es el tamaño máximo en bytes del contenido útil dentro de cada paquete.
- **ENCABEZADO_FORMAT = "!I":** Es el formato del encabezado del paquete. Se usa para convertir el número de secuencia (número de paquete) a bytes.
- **ENCABEZADO_SIZE = 4:** Es el tamaño en bytes del encabezado del paquete.

- **recibeCancion_gbn(sock: socket.socket, nombre_cancion: str, timeout=2.0, window_size=6)**

```python
def recibeCancion_gbn(sock: socket.socket, nombre_cancion: str, timeout=2.0, window_size=6) -> None:
    sock.sendto(f"GET:{nombre_cancion}".encode(), SERVER)
    sock.settimeout(5.0)

    try:
        data, _ = sock.recvfrom(65536)
    except socket.timeout:
        # Se considera que no hubo respuesta del servidor
        raise RuntimeError(">> Se excedio el tiempo de espera para la respuesta del servidor")

    encabezado = data.decode(errors="ignore")

    if (encabezado.startswith("ERROR")):
        raise RuntimeError(">> El servidor respondió con el error: " + encabezado)

    if (not encabezado.startswith("FILEINFO")):
        raise RuntimeError(">> Respuesta inesperada: " + encabezado)

    partes = encabezado.split("|")

    _, nombre_archivo, tamano_archivo_s, total_paquetes_s = partes
    tamano_archivo = int(tamano_archivo_s)
    total_paquetes = int(total_paquetes_s)

    print(f">> Se recibe del servidor: '{nombre_archivo}' ({tamano_archivo} bytes) en {total_paquetes} paquetes")

    paquetes = {}
    num_seq_esperado = 0

    sock.settimeout(timeout)
```

Imagen 8. Se obtienen los metadatos enviados por el servidor.

Descripción general: Esta función se encarga de enviar la solicitud al servidor para recibir la canción una vez que obtiene los paquetes, los usa para reconstruir la canción y poder reproducirla.

Parámetros:

- **sock : socket.socket**
  - Socket UDP ya creado y enlazado.
- **nombre_cancion : str**
  - Nombre de la canción elegida por el cliente.
- **timeout : float**
  - Tiempo limite para recibir respuesta del servidor.
- **window_size : int**
  - Número máximo de paquetes “en vuelo”.

En este bloque de código se envía la solicitud para recibir la canción. Intenta obtener la respuesta del servidor, si el tiempo de espera expira entonces se muestra el error. Si esto no sucede de la respuesta del servidor se extrae el encabezado para revisar si hubo un error en el envío o si no hay información del archivo. Si no se cumplen ambos casos, entonces del encabezado se extrae el nombre de la canción, el tamaño del archivo y el total de paquetes recibidos, para poder usarlos como referencia para después recibir los paquetes.

```python
    # Bucle donde se reciben los paquetes desde el servidor
    while (num_seq_esperado < total_paquetes): # Mientras no se reciban todos los paquetes
        try:
            paquete, _ = sock.recvfrom(4096)
        except socket.timeout:
            ACK_seq = num_seq_esperado - 1

            if (ACK_seq >= 0):
                ACK_paquete = b"ACK" + struct.pack(ENCABEZADO_FORMAT, ACK_seq)
                sock.sendto(ACK_paquete, SERVER)

            print(f">> No se recibió el paquete: Se reenvia ACK{ACK_seq} del último paquete confirmado")

            continue

        if (paquete == b"FIN"):
            break

        # Se verifica el tamaño mínimo del paquete
        if (len(paquete) < ENCABEZADO_SIZE):
            continue
```

Imagen 9. Guardado y confirmación de paquetes.

En este bloque de código es en donde se reciben los paquetes. Si el número de secuencia esperado es menor al total de paquetes, entonces faltan paquetes por recibir, se siguen recibiendo paquetes mientras esta condición se cumpla.

Se intenta recibir el paquete enviado por el servidor. Si el tiempo de espera para recibirlo se excede entonces, se considera que el paquete se perdió y entonces envía el número de secuencia del último paquete confirmado al servidor para que este reenvíe los paquetes desde ese número. Si el paquete se recibe, entonces se verifica que tenga el tamaño mínimo.

```python
        num_seq = struct.unpack(ENCABEZADO_FORMAT, paquete[:ENCABEZADO_SIZE])[0]
        bytes_paquete = paquete[ENCABEZADO_SIZE:]

        # Lógica para la aceptación del paquete
        if (num_seq == num_seq_esperado):
            paquetes[num_seq] = bytes_paquete

            while (num_seq_esperado in paquetes):
                num_seq_esperado += 1

            # Se envía el número de secuencia del último paquete confirmado (ACK)
            ACK_enviar = num_seq_esperado - 1
            ACK_paquete = b"ACK" + struct.pack(ENCABEZADO_FORMAT, ACK_enviar)
            sock.sendto(ACK_paquete, SERVER)

            print(f">> Se recibe y se envía ACK{ACK_enviar}")
        elif (num_seq > num_seq_esperado):
            if (num_seq not in paquetes):
                paquetes[num_seq] = bytes_paquete

            # Se reenvia el número de secuencia del último paquete confirmado (válido) (ACK)
            ACK_seq = num_seq_esperado - 1

            if (ACK_seq >= 0):
                ACK_paquete = b"ACK" + struct.pack(ENCABEZADO_FORMAT, ACK_seq)
                sock.sendto(ACK_paquete, SERVER)
        else:
            ACK_paquete = b"ACK" + struct.pack(ENCABEZADO_FORMAT, num_seq)
            sock.sendto(ACK_paquete, SERVER)
```

Imagen 10. Guardado y confirmación de paquetes.

En este bloque de código se obtiene el número de secuencia del paquete recibido, así como el contenido en bytes del paquete. Si el número de secuencia es el esperado, entonces se

guarda el paquete con su número de secuencia y se envía la confirmación del paquete recibido al servidor. Si el número de secuencia esta fuera de orden, se verifica que no haya sido guardado previamente antes de guardarlo, para después envía la confirmación de este paquete. Por último, si el paquete no es el espero y tampoco esta fuera de orden, entonces está duplicado, por lo que se envía la confirmación del paquete duplicado.

```
119     # Se ensamblan los bytes en orden del archivo y se almacena temporalmente
120     bytes_archivo = bytearray()
121
122     for pack in range(total_paquetes): # Se concatenan en orden los chunks (paquetes) desde 0 hasta total_paquetes
123         chunk = paquetes.get(pack, b"")
124         bytes_archivo.extend(chunk)
125
126     bytes_archivo = bytes_archivo[:tamano_archivo] # Se recorta el exceso (último chunk)
127
128     # Se crea y guarda un archivo temporal
129     temporal = tempfile.NamedTemporaryFile(delete=False, suffix="." + Path(nombre_archivo).suffix.lstrip("."))
130     temporal.write(bytes_archivo) # Se escribe todo el comtenido del archivo
131     temporal.flush()
132     ruta_temporal = temporal.name
133     temporal.close()
134
135     print(f"\n>> Archivo guardado temporalmente en: {ruta_temporal}")
136
137     return ruta_temporal # Devuelve la ruta temporal
```

Imagen 11. Reconstrucción y guardado de la canción.

En este bloque de código una vez que se recibieron todos los paquetes, se toma cada uno para reconstruir el archivo para después guardarlo en un archivo temporal, en donde devuelve la ruta en donde se guardó.

- **main()**

```
121 def main():
122     # Se crea el socket UDP
123     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
124     sock.bind(("", 0))
125     sock.settimeout(5.0)
126
127     print(">> Se solicita la lista de canciones al servidor...\n")
128     lista_canciones = pedirLista(sock)
129
130     if (not lista_canciones): # Si no hay canciones
131         print("\n>> No hay canciones disponibles o no se pudó obtener la lista")
132
133         return
134
135     print("Canciones disponibles:\n")
136
137     for index, cancion in enumerate(lista_canciones, start=1):
138         print(f"{index}.- {cancion}")
139
140     opcion = input("\nEscribe el número de la canción a reproducir: ").strip()
141
142     if (not opcion.isdigit() or int(opcion) < 1 or int(opcion) > len(lista_canciones)):
143         print("\n>> Opción inválida")
144
145         return
146
147     cancion = lista_canciones[int(opcion) - 1] # Canción elegida
148
149     print(f"\n>> Solicitando la canción '{cancion}'...")
```

Imagen 12. Solicitar la lista de canciones.

En este bloque de código se crea el socket del cliente. En un principio solicita la lista de canciones y cuando las recibe las imprime y le solicita al cliente que elija una de ellas. Una vez elegida la canción, se guarda su nombre.

```
151     try: # Trata de obtener la ruta de la canción
152         ruta_temporal = recibeCancion_gbn(sock, cancion)
153     except Exception as e:
154         print("\n>> Error al recibir la canción: ", e)
155
156         return
157
158     print("\n>> Reproducción iniciada")
159
160     # Llama al reproductor
161     reproductor(ruta_temporal, cancion)
162
163 if __name__ == "__main__":
164     main()
```

Imagen 13. Llamar a reproductor().

Luego se llama a la función **'recibeCancion_bgn()'** para enviar la solicitud y recibir la ruta temporal en donde se guardo la canción y para que al final llame a la función **'reproductor()'**, para reproducir la canción.

3.- Archivo: Reproductor.py

- **reproductor(ruta_cancion: str, nombre_cancion: str)**

```
7   def reproductor(ruta_cancion: str, nombre_cancion: str) -> None:
8       def reproducir() -> None:
9           """
10          Reproduce el archivo .mp3.
11          """
12          pygame.mixer.music.play()
13          label_state.config(text="🎶 Reproduciendo canción...")
14
15      def pausar() -> None:
16          """
17          Pausa la reproducción del archivo .mp3.
18          """
19          pygame.mixer.music.pause()
20          label_state.config(text="Canción en pausa")
21
22      def continuar() -> None:
23          """
24          Continua la reproducción del archivo .mp3.
25          """
26          pygame.mixer.music.unpause()
27          label_state.config(text="Continuando canción...")
28
29      def detener() -> None:
30          """
31          Detiene la reproducción del archivo .mp3.
32          """
33          pygame.mixer.music.stop()
34          label_state.config(text="Canción detenida")
```

Imagen 14. Función para el control de la reproducción.

Descripción general: Esta función toma el archivo .mp3 en la ruta correspondiente, para posteriormente abrir la interfaz gráfica del reproductor y poder controlar la reproducción de la canción.

Parámetros:

- **ruta_cancion : str**
  - Ruta del archivo .mp3 a reproducir.
- **nombre_cancion : str**
  - Nombre de la canción elegida por el cliente.

En este bloque de código se definen varias funciones para controlar la reproducción de la canción, como para reproducir la canción, detener la canción, pausar la canción y continuar con la reproducción.

```
46      def ajustarVolumen(valor: int) -> None:
47          """
48          Ajusta el volumen de la canción.
49
50          Parameters
51          ----------
52          valor : int
53              Valor inicial del volumen de la canción.
54          """
55          volumen = float(valor)
56
57          pygame.mixer.music.set_volume(volumen)
58          label_volume.config(text=f"Volumen al {int(volumen * 100)}%")
59
60      try:
61          pygame.mixer.music.load(ruta_cancion)
62          archivoCargado = True
63      except Exception as e:
64          archivoCargado = False
65          print(f"Error al cargar el archivo: {e}")
```

Imagen 15. Función para ajustar el volumen y la carga de la canción.

En este bloque de código se define la función para controlar el volumen de la canción y también se intenta cargar el archivo temporal creado previamente, para su posterior reproducción.

```python
    ventana = tk.Tk()
    ventana.title("Reproductor MP3")
    ventana.geometry("400x300")
    ventana.resizable(False, False)

    if (archivoCargado):
        label = tk.Label(ventana, text=f"Canción cargada:\n{nombre_cancion}", wraplength=380)
    else:
        label = tk.Label(ventana, text="No se pudo cargar la canción", fg="red")

    label.pack(pady=10)
    label_state = tk.Label(ventana, text="Reproductor inactivo", fg="gray")
    label_state.pack(pady=5)

    # Se crean los botones
    frame_btns = tk.Frame(ventana)
    frame_btns.pack(pady=15)

    btn_play = tk.Button(frame_btns, text="▶ Reproducir", command=reproducir, state="normal" if archivoCargado else "disabled")
    btn_play.grid(row=0, column=0, padx=5)
    btn_pause = tk.Button(frame_btns, text="⏸ Pausar", command=pausar, state="normal" if archivoCargado else "disabled")
    btn_pause.grid(row=0, column=1, padx=5)
    btn_continue = tk.Button(frame_btns, text="▶ Continuar", command=continuar, state="normal" if archivoCargado else "disabled")
    btn_continue.grid(row=0, column=2, padx=5)
    btn_stop = tk.Button(frame_btns, text="⏹ Detener", command=detener, state="normal" if archivoCargado else "disabled")
    btn_stop.grid(row=0, column=3, padx=5)

    label_volume = tk.Label(ventana, text="Volumen al 50%")
    label_volume.pack(pady=5)

    controlVolumen = ttk.Scale(ventana, from_=0, to=1, orient="horizontal", value=0.5, command=ajustarVolumen, length=250)
    controlVolumen.pack()
    pygame.mixer.music.set_volume(0.5)

    # Ejecutamos la ventana del reproductor
    ventana.mainloop()
```

Imagen 16. Creación y ejecución de la interfaz gráfica del .

Y por último en este bloque de código se crea la interfaz del reproductor para que se reproduzca la canción.

CONCLUSIÓN

- García Escamilla Bryan Alexis

En un principio el algoritmo Go-Back-N pareció ser muy complicado de entender, pero, sin embargo, no fue así. Lo que si fue complicado fue la construcción de los paquetes que el servidor envía al cliente, ya que cada uno se compone de un encabezado y del contenido como tal, así como la extracción del contenido de este en la parte del cliente. El encabezado contiene el número de secuencia del paquete a enviar y el contenido del paquete es el fragmento del archivo .mp3 a enviar, ambas cosas se convierten a bytes y al final se concatenan para de esta forma construir el paquete completo y posteriormente enviarse.

El cliente recibe el paquete y extrae el número de secuencia del encabezado y el contenido también para después guardarlo en un diccionario y al final reconstruir el archivo .mp3 y guardarlo como un archivo temporal.

- Meléndez Macedonio Rodrigo

Esta práctica fue esencial para entender mejor los sockets de datagramas, pues con estos hicimos un programa en Python funcional en el que el Servidor manda al Cliente una canción en formato mp3 divido en N paquetes mediante el uso del protocolo UDP para la transmisión de datos, sin embargo, este protocolo no asegura la transferencia de todos los datos enviados,

por lo que fue necesario el uso de un algoritmo de control de flujo llamado Retroceder N o Go-Back N (GBN) para asegurar la transferencia total de los paquetes. Este algoritmo de control de flujo es muy útil, pues en este caso se encarga de dividir la canción seleccionada por el Cliente en N paquetes de 1024 bytes hasta transferir la canción en su totalidad. Cada paquete recibido por el Cliente envía un mensaje de confirmación de vuelta al Servidor para asegurar que el paquete llego correctamente, en el orden y tiempo esperado, ya que en caso de que un paquete se pierda o demore durante la transferencia, el algoritmo se encargara de reenviar el paquete hasta que llegue al destino correctamente. Tras recibir todos los paquetes de bytes en el lado del Cliente, estos se van a juntar en orden en un archivo y ruta temporal en el que el Cliente podrá reproducir la música solicitada por él desde una interfaz gráfica que hicimos con el uso de la biblioteca “pygame” de Python. Por último, la práctica estuvo muy interesante y casi no tuve problemas al momento de realizar mi parte, a excepción del principio, ya que no me quedaba muy claro que es lo que se tenía que hacer respecto a la relación entre el Servidor y el Cliente. Pero con eso aclarado, la práctica salió bien y fue relativamente rápida de hacer.